

1. Introduzione

L'esercizio proposto ha lo scopo di esercitare la manipolazione delle liste in Python e la creazione di funzioni. È suddiviso in due parti: la prima riguarda il calcolo della lunghezza delle parole, la seconda la generazione di password.

2. Parte 1 – Lunghezza delle parole

2.1 Obiettivo

Creare una lista contenente alcune parole e generare una seconda lista che contenga la lunghezza di ciascuna parola.

2.2 Svolgimento

Nel file esercizioW7D1.py è stata definita una lista a con le parole: `a = [ "Ciao!", "Piacere!", "sono", "Miriana" ]`

Successivamente, è stata creata una lista b con la lunghezza di ogni parola usando una list comprehension:

```
b = [ len(x) for x in a ]
```

Questo tipo di lista permette di crearne una compatta e leggibile, partendo da un'altra sequenza come una lista, stringa o intervallo.

Infine, il risultato è stato stampato a terminale con: `print(b)`

2.3 Risultato

Il terminale ha mostrato: `[5, 8, 4, 7]` che corrisponde alle lunghezze delle parole nella lista a.

```
esercizioW7D1.py x
Documents > esercizioW7D1.py
1 # Scrivi una funzione che data in ingresso una lista A contenente n parole
2 # restituisca in output una lista B di interi che rappresentano la lunghezza delle parole contenute in A
3 a = [ "Ciao!", "Piacere!", "sono", "Miriana" ]
4 b = [ len(x) for x in a ]
5
6 print(b)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(kali@kali) - [~/Documents/gioco]
$ cd Documents
(kali@kali) - [~/Documents/gioco/Documents]
$ python3 esercizioW7D1.py
[5, 8, 4, 7]
(kali@kali) - [~/Documents/gioco/Documents]
$
```

3. Parte 2 – Generatore di password - Facoltativo

3.1 Obiettivo

Scrivere una funzione che generi una password casuale, scegliendo tra una versione semplice (solo lettere e numeri) e una complessa (con simboli).

3.2 Struttura del codice

Sono state definite due variabili con i caratteri disponibili:

```
ALFANUMERICI = string.ascii_letters + string.digits
```

```
TUTTI_ASCII = ALFANUMERICI + string.punctuation
```

La funzione `generate_password` riceve la lunghezza e il tipo di caratteri, e costruisce la password:

```
def generate_password(length: int, charset: str) -> str: password = [] for i in range(0, length): letter = random.choice(charset) password.append(letter) return "".join(password)
```

3.3 Interazione con l'utente

L'utente sceglie il tipo di password:

```
scelta = input("Complessa o semplice? C/S: ")
```

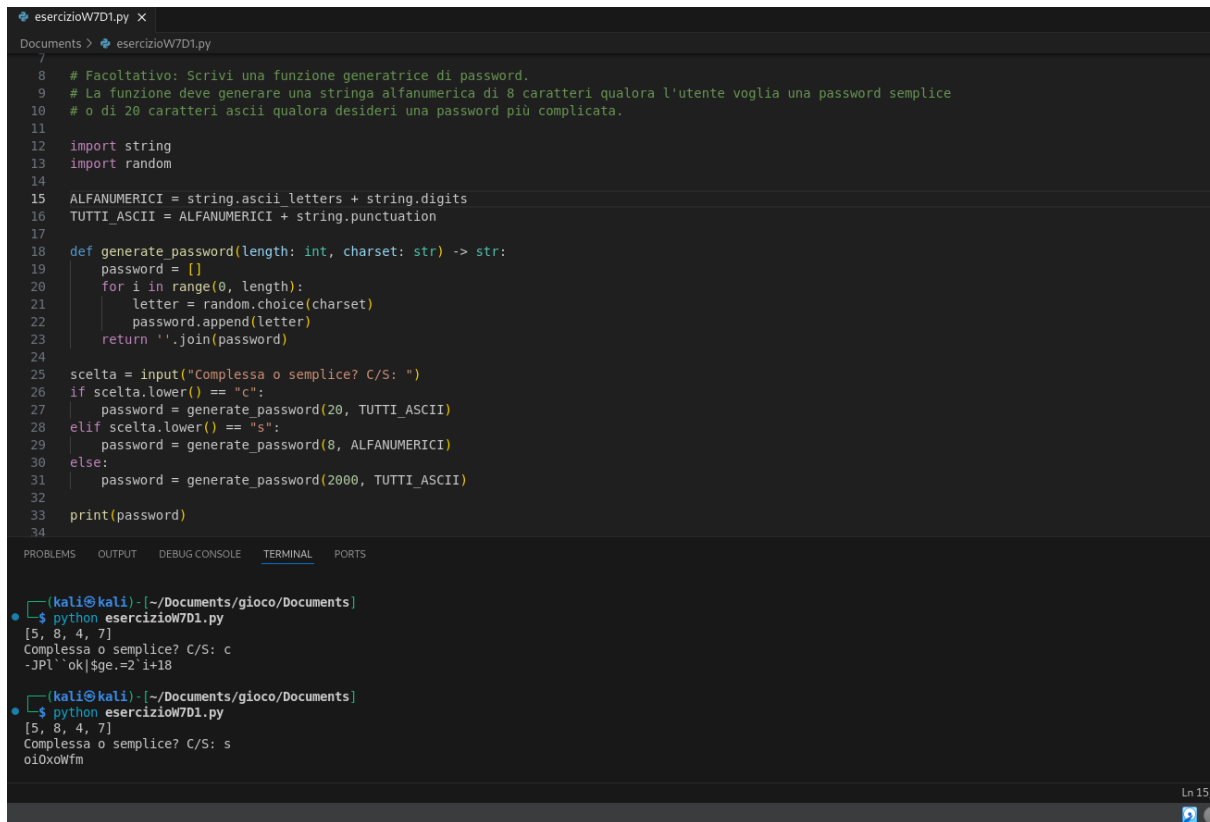
In base alla risposta, viene generata:

- una password di 8 caratteri se semplice
- una di 20 caratteri se complessa
- una di 2000 caratteri se l'input è errato

3.4 Esecuzione

Il file è stato eseguito da terminale con: `python3 esercizioW7D1.py`

E il risultato è stato stampato direttamente.



The screenshot shows a code editor with a file named `esercizioW7D1.py`. The script defines a `generate_password` function that takes `length` and `charset` as arguments. It uses `string` and `random` modules. The script prompts the user to choose between a complex password (C) or a simple password (S). If 'C' is chosen, it generates a 20-character password using all ASCII characters. If 'S' is chosen, it generates an 8-character password using alphanumeric characters. Otherwise, it generates a 2000-character password using all ASCII characters.

```
7
8 # Facoltativo: Scrivi una funzione generatrice di password.
9 # La funzione deve generare una stringa alfanumerica di 8 caratteri qualora l'utente voglia una password semplice
10 # o di 20 caratteri ascii qualora desideri una password più complicata.
11
12 import string
13 import random
14
15 ALFANUMERICI = string.ascii_letters + string.digits
16 TUTTI_ASCII = ALFANUMERICI + string.punctuation
17
18 def generate_password(length: int, charset: str) -> str:
19     password = []
20     for i in range(0, length):
21         letter = random.choice(charset)
22         password.append(letter)
23     return ''.join(password)
24
25 scelta = input("Complessa o semplice? C/S: ")
26 if scelta.lower() == "c":
27     password = generate_password(20, TUTTI_ASCII)
28 elif scelta.lower() == "s":
29     password = generate_password(8, ALFANUMERICI)
30 else:
31     password = generate_password(2000, TUTTI_ASCII)
32
33 print(password)
34
```

The terminal output shows two runs of the script. In the first run, the user enters 'c' and a 20-character password is generated. In the second run, the user enters 's' and an 8-character password is generated.

```
(kali@kali) ~/Documents/gioco/Documents
$ python3 esercizioW7D1.py
[5, 8, 4, 7]
Complessa o semplice? C/S: c
-JPl``ok|$ge.-2`i+18

(kali@kali) ~/Documents/gioco/Documents
$ python3 esercizioW7D1.py
[5, 8, 4, 7]
Complessa o semplice? C/S: s
oi0xoWfm
```

4. Conclusione

L'esercizio ha permesso di consolidare l'uso delle liste, delle funzioni e dell'interazione con l'utente. Inoltre, ha introdotto l'uso di moduli come `string` e `random` per operazioni più avanzate.